VNU-HCM

University of Science

Faculty of Information Technology

Computer Science

MyCraft

Project: Mycraft

Subject: Programming System

Students:

Nguyen Le Quoc Huy

(24125100)

Nguyen Hong Tan Tai

(24125078)

Supervising Teachers:

Ho Tuan Thanh

Do Nguyen Kha

August 23, 2025

Contents

1 Introduction

1.1 Project Description

- MyCraft is a 3D game like Minecraft game using C++. This is open world style. In MineCraft, players explore a procedurally generated, three-dimensional world with virtually infinite terrain made up of voxels. Players can discover and extract raw materials, craft tools and items, and build structures, earthworks, and machines. Depending on the game mode, players can fight hostile mobs, as well as cooperate with or compete against other players in multiplayer.
- Player will control a their character to survival, build their house and hunt the mobs.

1.2 Detail Desciption

- Player Charactor:
  - Behaviour:

* Break block: Player charactor will crack one block and obtains this block into inventory.

* Place block: Player charactor push one block at hover position.

Figure 1: Enter Caption

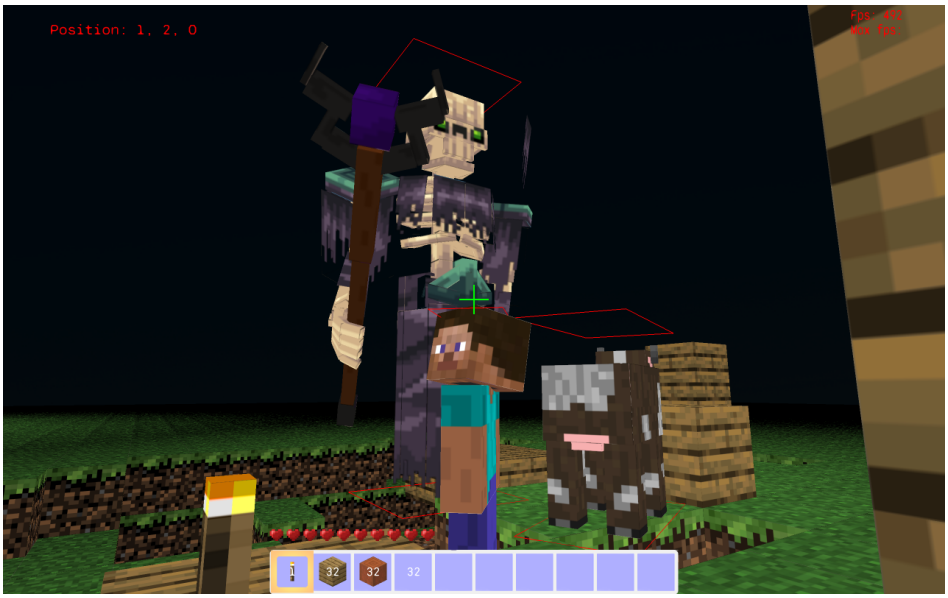
* Move: Use WASD to move following the camera direction.
* Jump: Use Space to jump.
* Attack: Damage one entity, such as zombie, pig, cow,...

    * Crouch: Press shift and move slower.
    * Run: Press Ctrl and run faster.
  - Healbar
  - Toolbar: Have 10 slots for 10 tools or block.
  - Inventory: Have 30 slots total, use for contains tools or blocks, has 2x2 crafting slots.

- Mobs:
  - Behviour: Move around one area, run away when hurt
  - Pig:

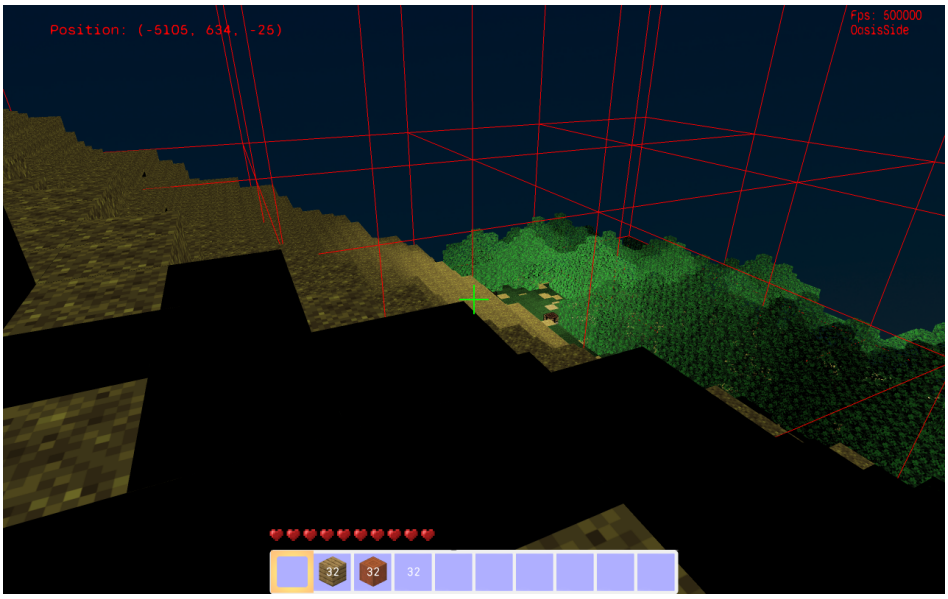
– Cow:

– Skeleton:



– Zombie:

- Light indensity contribution: Map will light some position has touch or other light object.



- Crafting Table: A crafting block has 3x3 craft block, can craft many block with specified recipe.

- Water: A fluid block can spread around, push entity move in it, player and some mobs can swim in it.

2 Libraries

2.1 C++ built-in library

2.2 OpenGL 4.6

OpenGL (Open Graphics Library) is a cross-language, cross-platform application programming interface (API) for rendering 2D and 3D vector graphics. The API is typically used to interact with a graphics processing unit (GPU), to achieve hardware-accelerated rendering. Being a low-level graphics api, is also the base library of many libs like raylib.

2.3 GLFW

GLFW is an Open Source, multi-platform library for OpenGL, OpenGL ES and Vulkan development on the desktop. It provides a simple API for creating windows, contexts and surfaces, receiving input and events. Being also the base library of raylib.

2.4 Glad

Supported library to load OpenGL on multiplatform

2.5 GLM

Supported library for OpenGL to calculate mathematics on 3D dimensions.

2.6 Bass

Provide adding sound feature.

3 Multi-platform adaption

3.1 Window:

We already test on Visual Studio Code with mingw32-make, cmake and gcc from MSYS2.

Required for OpenGL4.6.

3.2 Linux:

We already test on Ubuntu 24.04.2 LTS with Visual Studio Code. Required for OpenGL4.6 .

4 Application of Design Pattern

4.1 Singleton

4.1.1 Overview

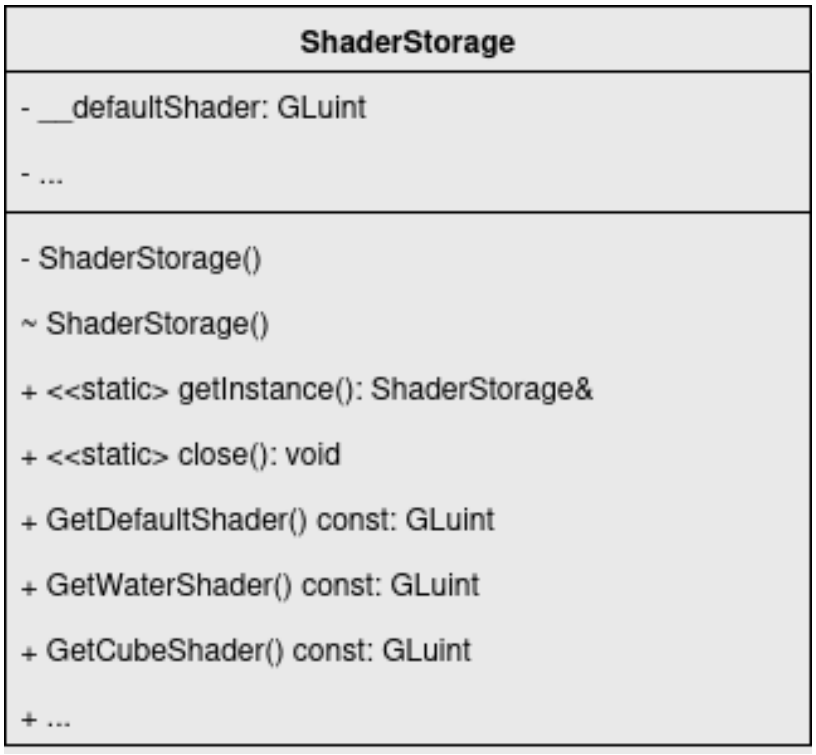
The Singleton pattern ensures that a class has only one instance throughout the program and provides a global access point to it.

Use cases:

- **Resource Control:** images, textures, sets of points/vertices, shaders.
- **GUI Management:** central control of UI components (often combined with Flyweight).
- **System Adapter:** wrapping OS functions for consistent access.
- **Event Manager:** handling and dispatching user input or system events.

Benefits:

- Guarantees single access to shared resources.
- Prevents conflicts and redundant objects.
- Replaces unsafe global variables with a clean design.



4.1.2 Implementation

ShaderStorage applies the Singleton pattern, ensuring only one global instance exists.

The constructor and destructor are private, preventing direct creation or deletion.

Access is controlled via getInstance(), while close() allows safe cleanup.

Stores all shader program IDs (default, water, cube, model, etc.) in one place.

Prevents duplication, ensures consistency, and simplifies shader resource management.

Provides a unified and maintainable way to handle GPU shaders across the system.

```
1
2  class ShaderStorage {
3      public:
4          static ShaderStorage& getInstance();
5          static void close();
6          ...
7      private:
8          ShaderStorage();
9          ~ShaderStorage();
10         static ShaderStorage* Default;
11         ...
```

```
12      };
13  }
```

4.1.3 Benefits:

- Centralized resource management – shaders, textures, and models are stored in one place, avoiding duplication.
- Global access point – any system (renderer, GUI, physics, etc.) can access shared resources easily.
- Consistency – ensures all components use the same shader programs, preventing mismatches.
- Memory efficiency – avoids creating multiple instances of heavy resources like shaders or textures.
- Controlled lifecycle – initialization and cleanup happen in one controlled instance (getInstance(), close()).
- Simpler design – removes the need for passing resource managers through many function calls.

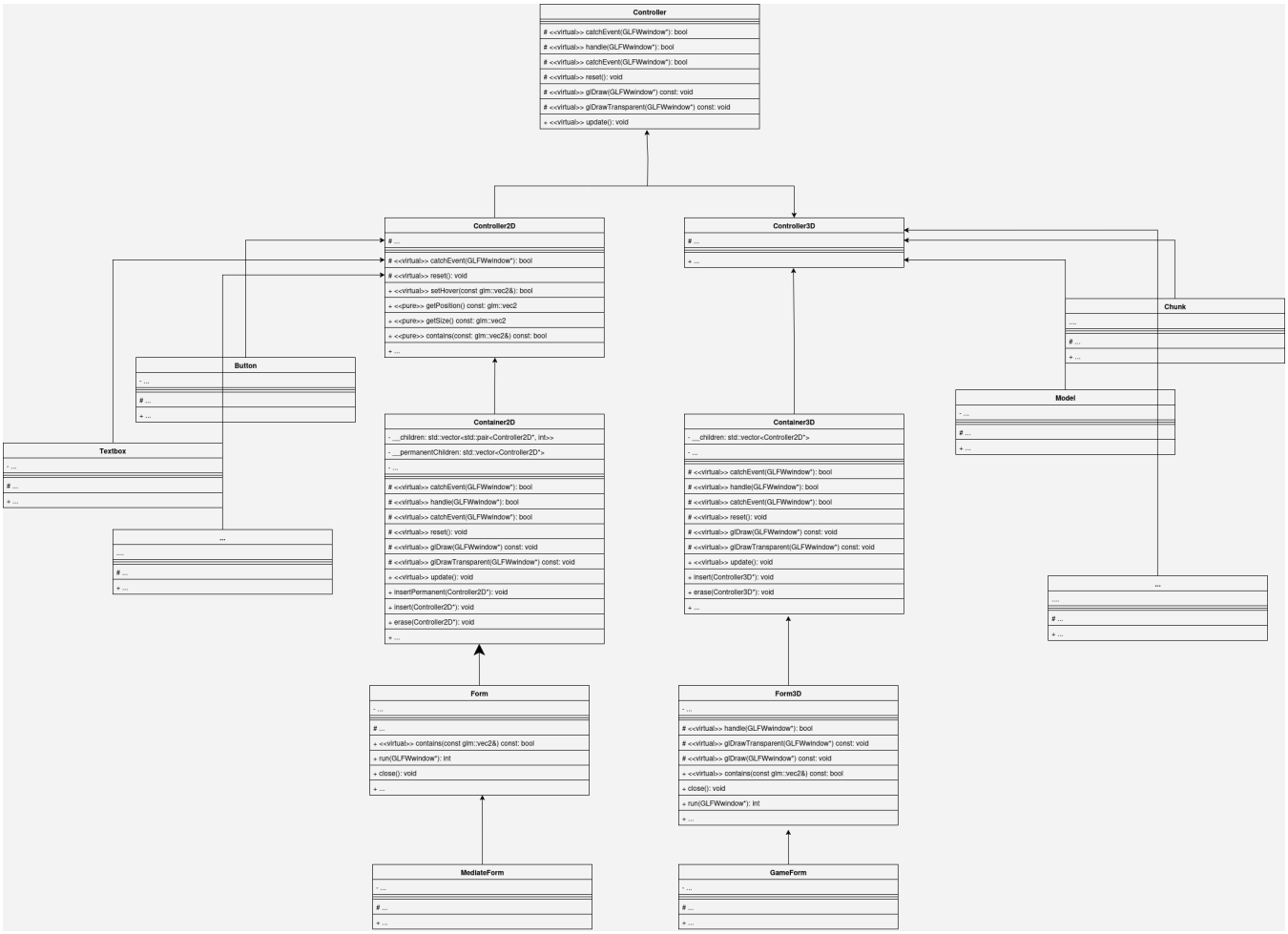
4.2 Composite:

4.2.1 Overview:

The **Composite pattern** is a fundamental structural pattern used to represent part-whole hierarchies and to treat individual objects and groups of objects uniformly. In GUI systems, this is especially important because every element can be considered a controller.
For example:

- Basic GUI components such as buttons, textboxes, and labels act as controllers.
- A container is also a controller, but it can hold multiple child controllers.
- A form is a special type of container, capable of managing both other containers and individual controls.

- The container delegates drawing and event handling to its child controllers, allowing complex interfaces to be composed from simpler elements.



4.2.2 Implementation:

4.2.3 Benefits:

- Uniformity – treat both single controls (button, textbox, label) and groups (container, form) in the same way.
- Hierarchy – supports building complex GUIs from nested elements easily.
- Reusability – containers and forms can be reused or extended without changing child controllers.
- Simplified event handling – containers delegate input and events to child elements automatically.

- Scalability – new components can be added without affecting the overall structure.
- Maintainability – clear part-whole structure makes the GUI system easier to manage and extend.

4.3 Decorator:

4.3.1 Overview:

The Decorator pattern allows attaching additional responsibilities or behaviors to an object dynamically, without modifying its original class. In this project, bag and toolbar is considered as two of inventory, so the Inventory will swap it core between toolbar(normal) and invenroty(when opening)

4.3.2 Implementation in the Project

The class `Inventory` acts as the main entry point for the inventory system:

```
1
2     class Wrapper: public MyBase::Controller2D {
3     public:
4         Wrapper(Controller2D* controller);
5         ~Wrapper();
6         void changeState(Controller2D* controller);
7         ...
8     protected:
9         virtual bool contains(const glm::vec2& position) const  override;
10        virtual bool catchEvent(GLFWwindow* window) override;
11        virtual bool handle(GLFWwindow* window) override;
12        virtual void glDraw() const override;
13        virtual void glDrawTransparent() const override;
14    protected:
15    private:
16        Controller2D* __controller;
17    };
18    class Inventory: public MyBase::Wrapper, public MyBase::Port {
19    public:
20        Inventory();
21        ~Inventory();
```

```
    private:
    };
```

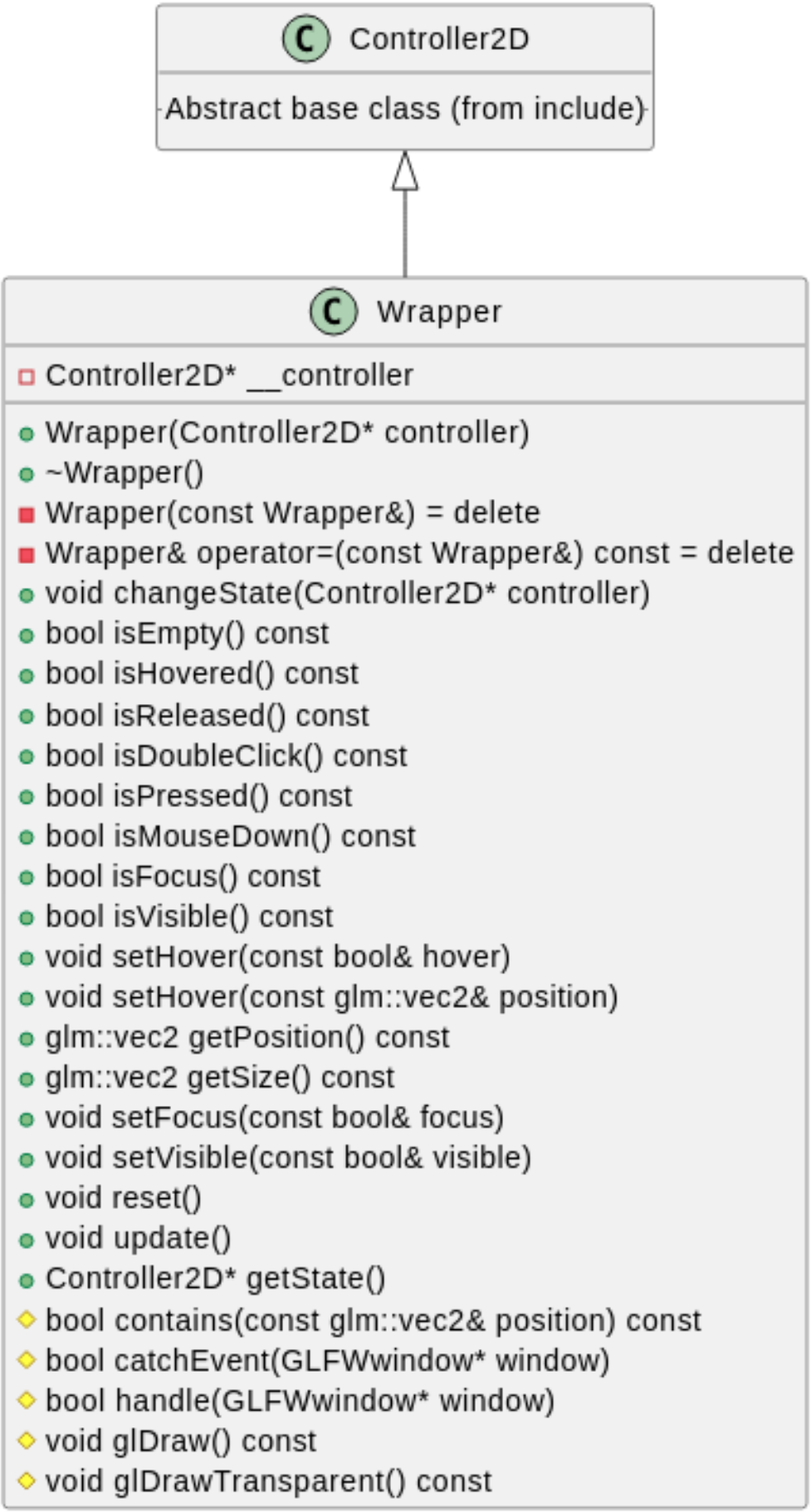
Here, the `Inventory` object holds a pointer to the abstract interface `InventoryUI`. This interface can be decorated (replaced) at runtime with different concrete implementations such as `ToolBar` or `Bag`.

- By default, current pointer points to a ToolBar object.
- When the player presses the **E** key, this pointer is reassigned to a `Bag` object.
- All calls such as `glDraw()`, `setHover()`, or `catchEvent()` are automatically forwarded to whichever implementation is currently active.

This behavior is possible because both `ToolBar` and `Bag` inherit from the common base class `InventoryUI` and override its virtual methods:

```

class InventoryUI: public MyBase::Container2D {
public:
    InventoryUI() = default;
    virtual ~InventoryUI();
    InventoryUI(const InventoryUI&) = delete;
    InventoryUI& operator=(const InventoryUI&) const = delete;

    virtual void close() = 0;
    virtual void open() = 0;
protected:
private:
};
class Bag: public InventoryUI {
    ...
    void glDraw() const override;
    void open() override;
    void close() override;
};

class ToolBar: public InventoryUI, public MyBase::Port {
```

```
22      ...
23      void glDraw() const override;
24      void open() override;
25      void close() override;
26  };
```


Controller2D
Abstract base class (from include)
Wrapper
Controller2D* __controller
Wrapper(Controller2D* controller)
~Wrapper()
Wrapper(const Wrapper&) = delete
Wrapper& operator=(const Wrapper&) const = delete
void changeState(Controller2D* controller)
bool isEmpty() const
bool isHovered() const
bool isReleased() const
bool isDoubleClick() const
bool isPressed() const
bool isMouseDown() const
bool isFocus() const
bool isVisible() const
void setHover(const bool& hover)
void setHover(const glm::vec2& position)
glm::vec2 getPosition() const
glm::vec2 getSize() const
void setFocus(const bool& focus)
void setVisible(const bool& visible)
void reset()
void update()
Controller2D* getState()
bool contains(const glm::vec2& position) const
bool catchEvent(GLFWwindow* window)
bool handle(GLFWwindow* window)
void glDraw() const
void glDrawTransparent() const

4.3.3 Benefits:

- Flexibility – new behaviors can be added to controllers at runtime without changing their code.
- Reusability – common shapes are shared, while decorations provide distinct functionality.
- Separation of concerns – keeps core controller logic clean, with extra features layered on top.
- Extensibility – easy to introduce new GUI features (e.g., hover effects, borders, shadows).
- Consistency – same base shape can serve multiple controls while still supporting unique behaviors.

4.4 Facade

4.4.1 Overview.

The **Facade design pattern** provides a simplified, unified interface to a complex subsystem. Instead of exposing all the details of the BASS audio library, the `Sound` class acts as a facade, wrapping the low-level C API calls into an easy-to-use object-oriented interface. This allows developers to work with sound playback through simple methods without needing to understand the full complexity of BASS.

4.4.2 Implementation.

The `Sound` class encapsulates initialization and playback of audio streams. Its constructor loads a file and creates a playback channel, while the methods `play()`, `pause()`, and `stop()` internally call the appropriate BASS functions. This provides a clean and minimal interface for audio control.

```
1 class Sound{
2     public:
3         Sound(const std::string & filename);
4         ~Sound();
5         void play();
6         void pause();
7         void stop();
8     private:
```

```
        unsigned int channel;
        static bool audio_device;
};
```

4.4.3 Benefits.

- **Simplification:** hides the complexity of the BASS API behind a clean interface.
- **Ease of use:** developers can control sound with simple, intuitive methods.
- **Encapsulation:** reduces dependency on the external library's details.
- **Maintainability:** changes in the audio library only require updates to the facade.
- **Reusability:** the same interface works for any sound file without exposing low-level code.

4.5 Flyweight, Bridge

4.5.1 Overview.

The **Flyweight** design pattern is used to minimize memory consumption by sharing common intrinsic state across multiple lightweight objects, while extrinsic state is kept externally by the client. This allows large numbers of similar objects to be represented efficiently. In this design, the classes `FlyWeightObject`, `FlyWeightCore`, and `FlyweightStorage` cooperate to achieve resource sharing and lifetime management.

On the other hand, Bridge is also applied to this situation, class FlyweightStorage cant be access directly, it must be control through FlyweightCore and FlyweightObject.

Moreover, Trie structure also using combined with FlyWeight, it controls there are only one data with specified link load once time. FlyweightCore is also prevent from copy assignment and only initial through FlyweightStorage.

4.5.2 Implementation.

```

class FlyWeightObject {
public:
    FlyWeightObject();
```

```
    FlyWeightObject(const FlyWeightObject& obj);
    FlyWeightObject(FlyWeightObject&& obj);
    FlyWeightObject& operator=(const FlyWeightObject& obj);
    virtual ~FlyWeightObject();
    bool isEmpty() const;
    void load(const std::string& src);
    const std::string& getSource() const;

    friend class FlyweightStorage;
protected:
    FlyWeightCore* getCore() const;
private:
    FlyWeightCore* __core;
    virtual FlyWeightCore* create(const std::string& src) const = 0;
};
```

The class `FlyWeightObject` acts as a handle for clients. It stores a pointer to a shared `FlyWeightCore` and provides methods such as `load`, `getSource`, and copy/move operators. The pure virtual `create()` enforces subclasses to define how a new core is built for a given resource.

```

class FlyWeightCore {
public:
    FlyWeightCore(const FlyWeightCore&) = delete;
    FlyWeightCore& operator=(const FlyWeightCore&) const = delete;
    friend class FlyWeightObject;
    friend class FlyweightStorage;
    virtual ~FlyWeightCore();
protected:
    FlyWeightCore();
private:
    unsigned int    *__count;
    std::string     __source;

    static void close(FlyWeightCore* core);
};
```

The class `FlyWeightCore` contains the intrinsic state (e.g., the resource data and its `__source`). It

also keeps a reference counter (`__count`) so that the storage knows when to release it. Copying is disabled to avoid duplicating heavy data.

```

class FlyweightStorage {
public:
    friend class FlyWeightObject;
    friend class FlyWeightCore;
    static void close();
protected:
private:
    static void getObject(FlyWeightObject& object, const std::string& name);
    static void removeObject(const std::string& name);
    static void create();
    struct TrieNode {
        TrieNode();
        unsigned int size;
        FlyWeightCore* obj;
        TrieNode* node[128];
    };
    TrieNode* __root;
    FlyweightStorage();
    ~FlyweightStorage();
    static FlyweightStorage* Default;
    ...
};
```

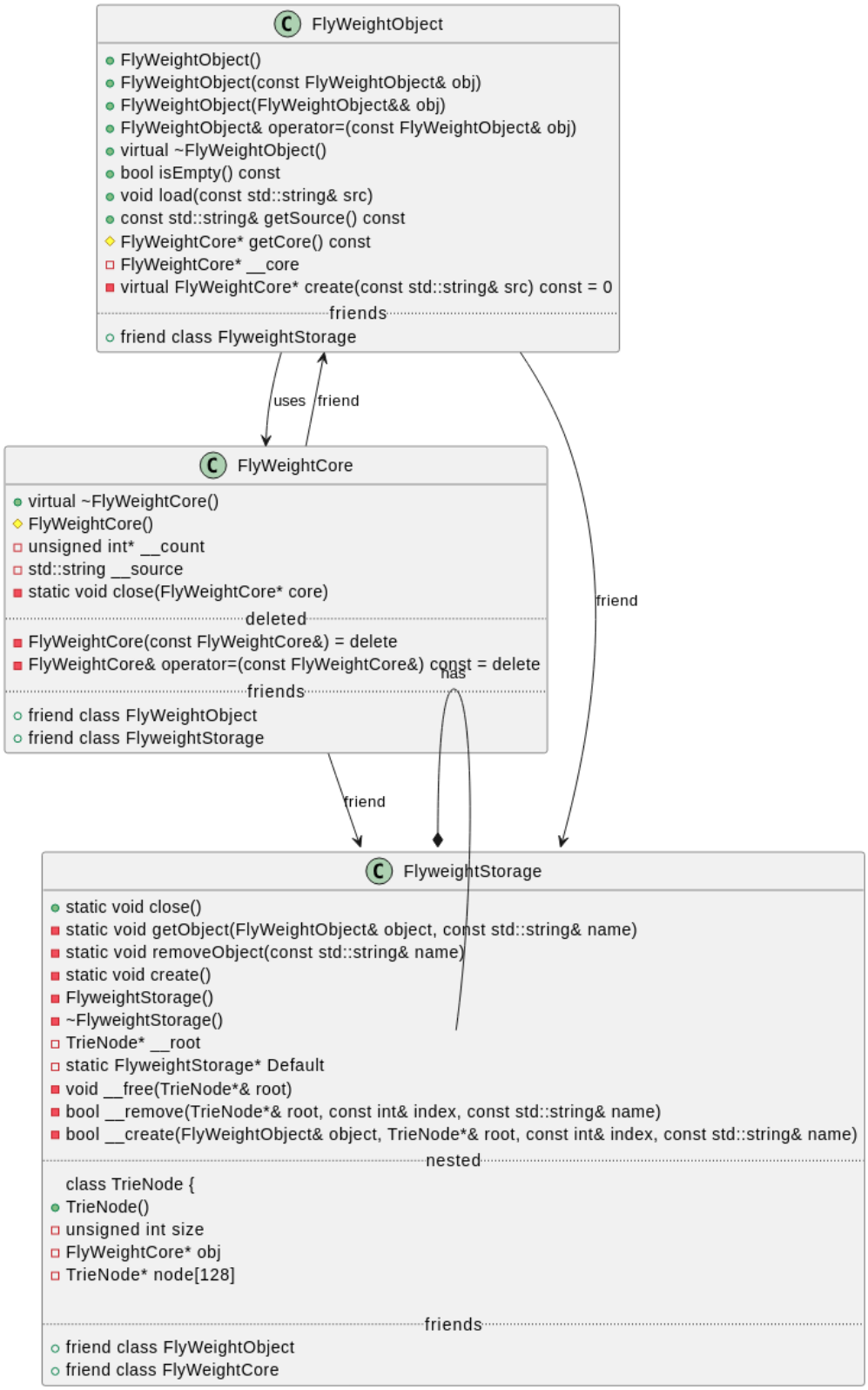
The class `FlyweightStorage` is responsible for indexing and managing cores. It uses a trie structure (`TrieNode`) to store objects by their string key. When `getObject` is called, it either retrieves an existing core or creates a new one via `FlyWeightObject::create`. It also provides cleanup functions to remove or free cores when reference counts drop to zero.

- `FlyWeightObject`
  - Holds a pointer `__core` to shared data.
  - Copy constructor and assignment operator increase the reference count.
  - Move constructor transfers ownership.

  - Destructor decreases the reference count and may trigger cleanup.
  - The abstract method `create()` lets subclasses define how resources are loaded.

- `FlyWeightCore`
  - Stores intrinsic state (`__source`).
  - Maintains reference count through `__count`.
  - Provides static method `close` to release memory when the count reaches zero.
  - Disables copying to ensure only one instance of each resource exists.

- `FlyweightStorage`
  - Acts like a singleton with static member `Default`.
  - Uses a trie to store resources by name with efficient lookup.
  - `getObject`: retrieves existing cores or creates new ones.
  - `removeObject` and `__remove`: reduce references and free nodes.
  - `close`: cleans up the entire storage when no longer needed.


FlyWeightObject
FlyWeightObject()
FlyWeightObject(const FlyWeightObject& obj)
FlyWeightObject(FlyWeightObject&& obj)
FlyWeightObject& operator=(const FlyWeightObject& obj)
virtual ~FlyWeightObject()
bool isEmpty() const
void load(const std::string& src)
const std::string& getSource() const
FlyWeightCore* getCore() const
FlyWeightCore* __core
virtual FlyWeightCore* create(const std::string& src) const = 0
friends
friend class FlyweightStorage
uses
friend
FlyWeightCore
virtual ~FlyWeightCore()
FlyWeightCore()
unsigned int* __count
std::string __source
static void close(FlyWeightCore* core)
deleted
FlyWeightCore(const FlyWeightCore&) = delete
FlyWeightCore& operator=(const FlyWeightCore&) const = delete
friends
friend class FlyWeightObject
friend class FlyweightStorage
friend
has
friend
FlyweightStorage
static void close()
static void getObject(FlyWeightObject& object, const std::string& name)
static void removeObject(const std::string& name)
static void create()
FlyweightStorage()
~FlyweightStorage()
TrieNode* __root
static FlyweightStorage* Default
void __free(TrieNode*& root)
bool __remove(TrieNode*& root, const int& index, const std::string& name)
bool __create(FlyWeightObject& object, TrieNode*& root, const int& index, const std::string& name)
nested
class TrieNode {
TrieNode()
unsigned int size
FlyWeightCore* obj
TrieNode* node[128]
friends
friend class FlyWeightObject
friend class FlyWeightCore

4.5.3 Benefits.

- **Memory efficiency:** identical resources (textures, sounds, models, etc.) are loaded only once and shared.
- **Performance:** avoids redundant loading of the same data, reducing startup or runtime overhead.
- **Centralized lifecycle management:** automatic reference counting ensures resources are freed only when unused.
- **Extensibility:** new resource types can be supported simply by subclassing `FlyWeightObject` and implementing `create`.
- **Consistency:** all handles pointing to the same source are guaranteed to reference the same underlying data.

4.6 Proxy

4.6.1 Overview.

The **Proxy** pattern provides a surrogate object that controls access to a heavier, real object. In this voxel world, `Chunk` is heavy (blocks, water, lighting, GPU data), so the system introduces *two layers of proxies* to defer loading, centralize access, and keep client code independent from the concrete storage and lifetime of chunk data.

4.6.2 Implementation

Layer 1: Interface Proxy (`ChunkObject` → `Chunk`). `ChunkObject` is an abstract interface that declares all operations needed on a chunk (`getType`, `setType`, `setLight`, `pourWater`, `getState`, etc.). Client code talks to `ChunkObject`, not to the concrete `Chunk`. The heavy class `Chunk` (the real subject) implements these operations and stores the actual data.

```
1
2   class ChunkObject {
3   public:
4       ...
```

```
5         virtual ~ChunkObject();
6         virtual bool isInWater(const glm::ivec3& position) const = 0;
7         virtual const BlockCatogary& getType(const glm::ivec3&) const = 0;
8         virtual const BlockCatogary& getType(const glm::ivec3&) = 0;
9         virtual void setType(const glm::ivec3&, const BlockCatogary& type) = 0;
10        virtual void setState(const glm::ivec3& position, const glm::mat4& state
    ) = 0;
11        virtual void setLight(const glm::ivec3& position, const float& indensity
    ) = 0;
12        virtual bool pourWater(const glm::ivec3& position, const glm::vec4&
    height) = 0;
13        virtual bool takeWater(const glm::ivec3& position) = 0;
14        virtual float getWaterHeight(const glm::ivec3& position) const = 0;
15        virtual void pushDynamicWater(const glm::ivec4& position) = 0;
16        virtual void popDynamicWater(const glm::ivec4& position) = 0;
17        virtual glm::mat4 getState(const glm::ivec3&) const = 0;
18        const BlockCatogary& getLocalType(const glm::ivec3&) const;
19        const BlockCatogary& getLocalType(const glm::ivec3&) ;
20        void setLocalType(const glm::ivec3&, const BlockCatogary& type) ;
21        void setLocalState(const glm::ivec3& position, const glm::mat4& state);
22        void enableLocalBit(const glm::ivec3&) ;
23        void disableLocalBit(const glm::ivec3&) ;
24        void setLocalLight(const glm::ivec3& position, const float& indensity);
25        virtual std::bitset<16>::reference getLocalBit(const glm::ivec3&);
26
27        virtual std::bitset<16>::reference getBit(const glm::ivec3&) = 0;
28        virtual void enableBit(const glm::ivec3&)    = 0;
29        virtual void disableBit(const glm::ivec3&)   = 0;
30    protected:
31        virtual const glm::ivec3& getPosition() const = 0;
32    };
```

Layer 2: Access/Virtual Proxy (`ChunkLoader`/`DynamicChunk` → `Chunk`). `ChunkLoader` derives from `ChunkObject` and adds lookup/routing responsibilities (`contains`, `getChunk`, water/-light helpers). Concrete loaders (e.g., `DynamicChunk`) hold or acquire a real `Chunk` on demand and forward calls. This enables lazy loading and centralized control (e.g., caching, eviction, visibility checks).

```
1
2    class ChunkLoader: public ChunkObject {
3    public:
4        ...
5        virtual bool isInWater(const glm::ivec3& position) const override;
6        virtual bool contains(const glm::ivec3& position) const = 0;
7        const BlockCatogary& getType(const glm::ivec3&) const   override;
8        const BlockCatogary& getType(const glm::ivec3&)         override;
9        virtual void setType(const glm::ivec3&, const BlockCatogary& type)
   override;
10       void setState(const glm::ivec3& pos, const glm::mat4& state) override;
11       bool pourWater(const glm::ivec3& position, const glm::vec4& height)
   override;
12       bool takeWater(const glm::ivec3& position) override;
13       float getWaterHeight(const glm::ivec3& position) const override;
14       glm::mat4 getState(const glm::ivec3&) const override;
15       std::bitset<16>::reference  getBit(const glm::ivec3&) override;
16       void enableBit(const glm::ivec3&) override;
17       void disableBit(const glm::ivec3&) override;
18       void pushDynamicWater(const glm::ivec4& position) override;
19       void popDynamicWater(const glm::ivec4& position) override;
20
21       void setLight(const glm::ivec3& position, const float& indensity)
   override;
22       void removeLight(const glm::ivec3& position);
23       virtual void placeDynamicWater(const glm::ivec4& position);
24       virtual Chunk& getChunk(const glm::ivec3&)         = 0;
25       virtual const Chunk& getChunk(const glm::ivec3&) const   = 0;
26       float getLightIndensity(const glm::ivec3& position) const;
27   private:
28       BvhLightTree tree;
29   };
```

Manager as Proxy Coordinator (`ChunkManage`). `ChunkManage` combines a 3D container and a loader; it tracks player position, maintains a window of active chunks, and routes rendering and updates. It uses loader semantics to decide which real chunks must exist in memory and which can be deferred or unloaded.

```
class ChunkManage: public MyBase3D::Container3D, public ChunkLoader {
public:
    ...
    void playerAt(const glm::ivec3& position);
    Chunk&          getChunk(const glm::ivec3&)         override;
    const Chunk&    getChunk(const glm::ivec3&) const   override;
    void placeDynamicWater(const glm::ivec4& position) override;
    const glm::ivec3& getPosition() const override;
private:
    ...
    WaterManage             __waterManage;
    bool handle(GLFWwindow* window) override;
    void glDraw() const override;
    void glDrawTransparent() const override;
    ...

};
```

```
class DynamicChunk: public ChunkLoader {
public:
    ...
protected:
    Chunk&          getChunk(const glm::ivec3&) override;
    const Chunk&    getChunk(const glm::ivec3&) const override;
private:
    ...
};
```

How calls flow.

1. Game systems call methods on `ChunkObject` (Layer 1 proxy interface).
2. If the receiver is a `ChunkLoader`/`DynamicChunk`, it acts as a virtual proxy: it locates/creates the target `Chunk` and forwards the operation (Layer 2).
3. The concrete `Chunk` (real subject) performs the heavy work on its in-memory data structures and rendering buffers.

4.6.3 Benefits.

- **Lazy loading & memory efficiency:** real chunks are created/loaded only when needed.
- **Decoupling:** client code depends on `ChunkObject` interface, not on `Chunk` internals.
- **Centralized access control:** loaders/managers can gate, cache, or evict chunks consistently.
- **Scalability:** easy to change loading policy (disk, network, procedural) without touching gameplay code.
- **Maintainability:** rendering, water/lighting updates, and storage strategies evolve behind the proxies.

4.7 Command, Mediator, Observe

4.7.1 Overview

In this module, three well-known design patterns are applied to manage communication between different game components: **Command**, **Mediator**, and **Observer**. Each pattern plays a different role:

- The **Command pattern** encapsulates game actions into objects, decoupling the message sender from the action executor.
- The **Mediator pattern** introduces a central component (`Network`) to handle communication between multiple `Port` objects, reducing direct dependencies.
- The **Observer pattern** allows `Command` objects to be notified when relevant `Message` types arrive at a `Port`.

—

4.7.2 Implementation

Command Pattern The `Command` class is the core of the Command pattern:

```
1    class Command {
2    public:
```

```
        Command();
        virtual MessageType getType()      const = 0;
        virtual void execute(Port& mine, Port& source, Message* message) = 0;
        virtual ~Command() = 0;
    };
```

- Each concrete subclass of `Command` implements `getType()` and `execute()`.
- The `Port` class maintains a mapping from `MessageType` to the corresponding `Command`:

```
    std::map<MessageType, Command*> __commands;

```

- When a `Message` arrives, the correct `Command` is invoked via `execute()`.

This design decouples the message structure (`Message`) from the execution logic (`Command`).

—

Mediator Pattern The `Network` class implements the Mediator pattern:

```
    class Network {
    public:
        static void match(Port* port);
        virtual void receive(Port& source, Message* Message);
        virtual void receive(Port& source, Port& destination, Message* Message);
        static void close();
    private:
        Network();
        ~Network();
        static Network& get();
        std::vector<std::vector<Port*>>             __ports;
        void send(Port& source, Port& destination, Message* Message);
        static Network* network;
    };
```

- `Network` is a central hub that coordinates message passing between different `Port` objects.
- Instead of `Port` objects communicating directly with each other, they send messages through `Network`.

- This reduces coupling and simplifies the management of communication channels.

For example:

```
void Network::receive(Port& source, Port& destination, Message* message);
```

Here the mediator decides how the message flows from one port to another.

—

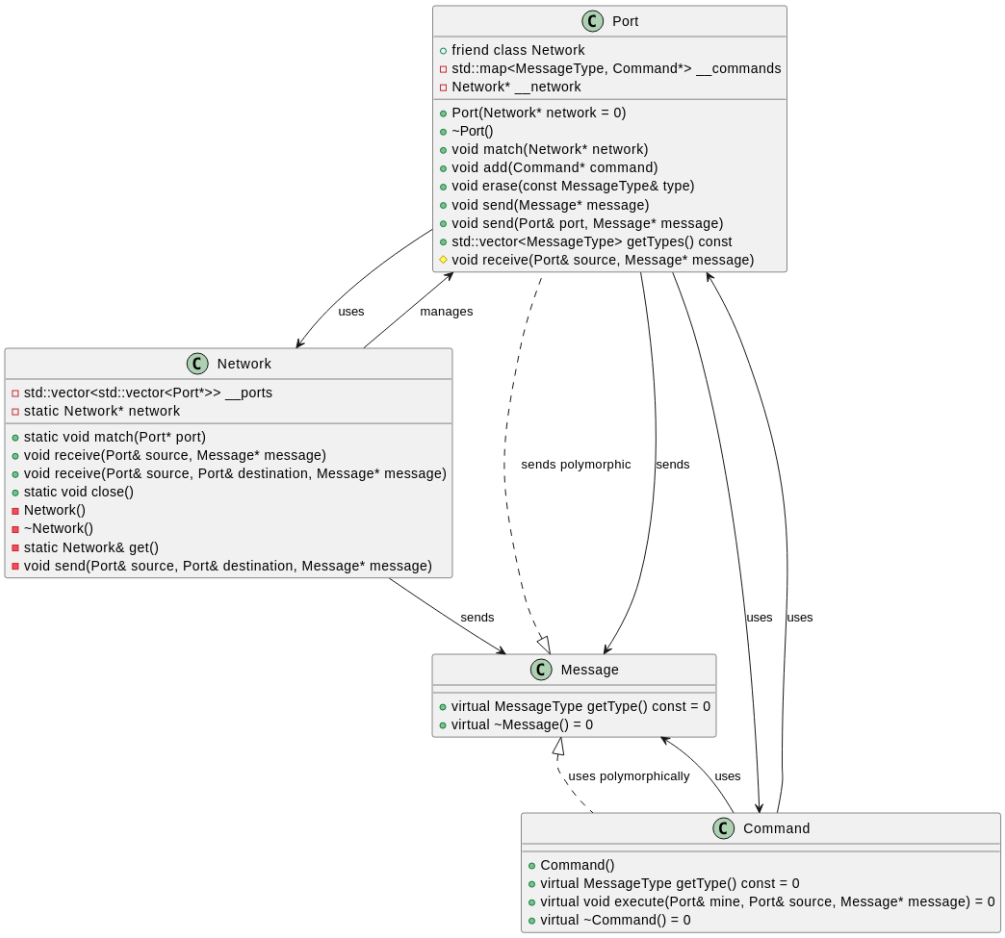
Observer Pattern The Observer pattern is applied through the interaction between `Port`, `Message`, and `Command`:

```
class Port {
public:
    Port(Network* network = 0);
    virtual ~Port();

    virtual void match(Network* network);

    void add(Command* command);
    void erase(const MessageType& type);
    void send(Message* Message);
    void send(Port& port, Message* Message);
    std::vector<MessageType> getTypes() const;
    friend class Network;
protected:
    virtual void receive(Port& source, Message* Message);
private:
    std::map<MessageType, Command*> __commands;
    Network* __network;
};
```

- A `Port` acts as a *Subject*, holding multiple `Command` observers.
- Each `Command` is registered for a specific `MessageType`.
- When a `Message` of that type arrives at the `Port`, the `Command` is notified via:

```
    virtual void receive(Port& source, Message* message);

```

Thus, the arrival of a new message triggers the execution of observers that have registered for that type.



—

4.7.3 Benefits

- **Command Pattern:**
  - Encapsulates actions, making it easy to add new commands without modifying existing code.
  - Separates message representation from execution logic.
- **Mediator Pattern:**
  - Centralizes communication logic, reducing direct dependencies between ports.

  - Improves maintainability and scalability of the networking system.

- **Observer Pattern:**

  - Enables event-driven architecture, where commands automatically react to messages.
  - Promotes flexibility since multiple commands can subscribe to different message types.

5 Data structure:

This project at the submit time uses 4 custom data stucture:

- Trie: Control loading resource, dont allow one link loaded to system more once time.

```
1
2       class FlyweightStorage {
3       public:
4           ...
5       private:
6           ...
7           struct TrieNode {
8               TrieNode();
9               unsigned int size;
10              FlyWeightCore* obj;
11              TrieNode* node[128];
12          };
13          ...
14      };
15
```

- Tree: Contains node of drawing meshes, draw meshes by hierrachy from top to down.

```
1       class GLTFStaticMesh: public MyBase::FlyWeightCore {
2       public:
3           ...
4       private:
5           struct Node {
6               std::vector<Node*> children;
7               glm::vec3 translation;
```

```
            int node;
            ~Node();
        };
        ...
    };

```

- Boundary Volumn Hierarchy under AVL tree: Using to manage and get light source, check hitbox

```
        class BvhLightTree {
        public:
            ...
            float getLightIndensity(const glm::vec3& position) const;
        private:
            struct Node {
                unsigned char height;
                float radius;
                glm::vec3 position;
                Node* right, *left;
                bool operator-(const Node& node);
                Node operator+(const Node& node);
                ...
            };
            Node* __root;
            ...
        };

```

```
    class HitboxNode {
    public:
        ...
        HitboxNode* parent, *left, *right;
        int height;
        bool contains(const glm::vec3& position) const;
        float isCollistion(const glm::vec3& postiion, const glm::vec3&
dir) const;
        virtual void setShape(const glm::mat4x3&);
```

```
            virtual glm::mat4x3 getShape() const;
            void update();
            float operator-(const HitboxNode& node) const;
            HitboxNode operator+(const HitboxNode& node) const;
            ...
        private:
            glm::mat4x3 __shape;
            glm::vec2   x, y, z;
            HitboxTree* tree;
        };

```

- Circle linked list: Use to store and get state of animation:

```
            struct Node {
                float       percent;
                glm::quat   state;
                Node*       next;
                Node(Node*);
                ~Node();
            };

```

6 Application of OOP Principles

In this project, the four fundamental principles of Object-Oriented Programming (OOP) — **Encapsulation**, **Inheritance**, **Polymorphism**, and **Abstraction** — are extensively applied to improve modularity, maintainability, and scalability of the system.

6.1 Encapsulation

Encapsulation is applied by grouping related data and behaviors into cohesive classes while restricting direct access to internal details through access specifiers (`private`, `protected`, `public`).
For example, in the `HashTable` class, the internal data structure (`buckets`, `size`) is kept private, while the public methods (`insert`, `find`, `remove`) provide a safe interface. Similarly, in `Message.h`,

the `Port` class encapsulates a mapping between `MessageType` and `Command`, hiding implementation details with a clean API:

```
private:
    std::map<MessageType, Command*> __commands;
    Network* __network;
```

Benefit: Encapsulation prevents unintended modification of internal state and enforces controlled interaction with objects.

6.2 Inheritance

Inheritance is used to promote code reuse and extend base functionalities.

For instance, `FlyWeightObject` is defined as an abstract base class, and different resources (e.g., textures, models, sounds) can inherit from it by implementing the `create()` method:

```
class FlyWeightObject {
    ...
    virtual FlyWeightCore* create(const std::string& src) const = 0;
};
```

Additionally, GUI components such as `Button`, `InputBox`, and `Form` inherit from a base `Controller`, reusing shared logic while allowing specialization.

Benefit: Inheritance reduces duplication and provides a structured class hierarchy, making extensions easier.

6.3 Polymorphism

Polymorphism allows objects of different types to be treated through a common interface.

For example, the `Command` class defines a polymorphic interface:

```
class Command {
public:
    virtual MessageType getType() const = 0;
    virtual void execute(Port& mine, Port& source, Message* message) = 0;
    virtual ~Command() = 0;
};
```

Concrete commands (e.g., `MoveCommand`, `JumpCommand`) override these methods, and the `Port` executes them without knowing the exact subclass type.

Similarly, in the animation system, the `Animation` base class provides virtual methods for fade-in/out, movement, and drawing, while each derived animation type implements its own behavior.

Benefit: Polymorphism enhances flexibility, allowing new behaviors to be added without modifying existing code.

6.4 Abstraction

Abstraction is applied by exposing only essential features while hiding complex implementation details.

For example, the `Sound` class acts as a Facade to the BASS audio library:

```
class Sound {
public:
    Sound(const std::string & filename);
    void play();
    void pause();
    void stop();
private:
    unsigned int channel;
    static bool audio_device;
};
```

Here, the user only needs to call `play()`, `pause()`, or `stop()`, without dealing with the low-level details of the audio engine.

Another example is the `Network` mediator, which abstracts the complexity of port-to-port communication, offering a clean interface for message passing.

Benefit: Abstraction reduces complexity, providing developers with clear and high-level APIs instead of low-level technical details.

—

7 Technical Problem

- **Research Challenges:** Choosing to work on a 3D game project this semester was a bold decision. With limited knowledge of 3D development and optimization techniques, maintaining a stable FPS proved to be a major challenge. Researching unfamiliar topics that we haven't learned before was especially difficult.
- **Inaccurate Documentation:** Some of the available documentation was incorrect or unclear. As a result, we had to experiment multiple times to achieve the desired outcome, which added to the difficulty of the project.
- **Multi-platform Challenge:** Since we used different computers and operating systems, making the project compatible across multiple platforms was not an easy task.
- **Time Constraints:** Time was another significant limitation. Despite the large scope of the project, we did our best to implement many core functions and features, even though some parts remain incomplete.
- **Future Improvements:** We hope to improve or refactor this project in the near future. Thank you for your attention and support!

A Appendix

- Design Pattern: https://refactoring.guru/
- : Github: https://github.com/Wanderer2414/RawMinecraft.git